

① アプリケーションの利用マニュアル

■ アプリケーション概要

本アプリケーションは、開発案件の申請、承認、進捗管理、予算管理を一元管理するためのWebアプリケーションです。

利用者は、申請者、部門管理者、本部管理者の3種類です。

■ ログイン方法

1. ログイン画面にアクセスします。
2. テストアカウント一覧に記載されたログインIDとパスワードを入力します。
3. ログイン後、ユーザーのロールに応じたトップ画面へ移動します。

■ 申請者の使い方

申請者は、新規案件の申請、申請状況の確認、承認済み案件の進捗・予算実績の更新を行います。

主な操作は以下のとおりです。

1. 申請者トップ画面で、自分の申請中案件や開発中案件を確認します。
2. 新規案件申請画面から、案件名、目的、担当部門、開始終了予定日、概算予算、工数を入力して申請します。
3. 申請状況確認画面で、部門承認待ち・本部承認待ち・却下などの状態を確認します。
4. 承認が済み開発中となった案件は、案件進捗管理画面からタスクの進捗や予算実績を更新します。

■ 部門管理者の使い方

部門管理者は、自部門の案件について、一次承認・却下、進捗確認、予算状況確認を行います。

主な操作は以下のとおりです。

1. 部門管理者トップ画面で、自部門の承認待ち案件や進行中案件を確認します。
2. 承認審査画面で、申請内容、予算、予定期間などを確認します。
3. 問題がなければ承認し、本部承認待ちへ進めます。
4. 修正が必要な場合は、却下理由を入力して却下します。
5. 案件モニタリング画面で、承認済み案件の進捗や予算状況を確認します。
6. 全タスクが完了している案件は、案件完了認定を行えます。

■ 本部管理者の使い方

本部管理者は、全部門の案件を横断して確認し、最終承認・却下を行います。

主な操作は以下のとおりです。

1. 本部管理者トップ画面で、全部門の案件状況や予算状況を確認します。
2. 最終承認待ち案件を開き、案件内容、部門、予算、予定期間などを確認します。
3. 問題がなければ最終承認します。
4. 修正が必要な場合は、却下理由を入力して却下します。
5. 最終承認された案件は、開発管理中の案件として進捗・予算管理の対象になります。

■ 通知機能について

承認、却下、完了認定など、確認が必要な操作が発生した場合は通知が表示されます。

通知は画面上部のベルアイコンから確認できます。未読通知がある場合は件数が表示され、通知ドロップダウンから直近の通知内容を確認できます。

なお、通知の件数や内容は、画面を再読み込みしたタイミングで最新状態に更新されます。そのため、別のユーザーが承認・却下などの操作を行った直後に通知が表示されない場合は、画面をリロードしてご確認ください。

② アプリケーションのデプロイ先

デプロイには、アプリケーション本体の公開先として Render、データベースとして Neon PostgreSQL を使用しています。

③ デプロイ手順

本課題の提出用リポジトリは企業GitLab上の非公開プロジェクトですが、Renderから企業GitLabへの直接連携が難しかったため、管理者の方針に沿って、Render連携用としてGitHubのPrivateリポジトリも使用しました。

ソースコードの提出・管理はGitLabを本流とし、Renderへの反映が必要なタイミングでGitHub Privateリポジトリの main ブランチにも反映する運用としています。

主な手順は以下のとおりです。

1. 企業GitLabで開発・確認を行い、Merge Request を作成して main ブランチへマージする。
2. ローカル環境で企業GitLabの main ブランチを最新化する。
`git switch main`
`git pull origin main`
3. Render連携用のGitHub Privateリポジトリへ、同じ main ブランチを反映する。
`git push github main`
4. Renderで新規 Web Service を作成し、GitHub Privateリポジトリ `dev-management-app` を連携する。
5. Renderの基本設定を以下のように設定する。
Name:`dev-management-app`
Branch:`main`

Language: Python 3

Region: Singapore

Build Command: `pip install -r requirements.txt`

Start Command: `gunicorn app:app`

6. Neonで作成したPostgreSQLの接続文字列を取得し、Renderの Environment Variables に以下を設定する。

`DATABASE_URL`: Neon PostgreSQL の接続文字列

`SECRET_KEY`: 本番環境用の秘密鍵

7. Render上でデプロイを実行し、Web Service が Live になることを確認する。
8. Render Shell でマイグレーション状態を確認し、必要に応じて適用する。

`flask --app app db current`

`flask --app app db heads`

`flask --app app db upgrade`

9. Render Shell からシードデータを投入する。

`python seed.py`

10. 本番URLにアクセスし、ログイン画面と各ロールの主要画面が表示できることを確認する。

■ デモデータの初期化運用について

動作確認しやすい状態を保つため、必要に応じて Render Shell から `seed.py` を実行し、デモデータを初期状態へ戻します。

実行コマンドは以下のとおりです。

`python seed.py`

この操作は、ソースコードの変更や再デプロイを伴わず、本番データベース上の動作確認用データを整えるためのものです。

課題管理者様へ確認したうえで、`push` を伴わない管理として実施します。

④ テストアカウント一覧

本アプリでは、以下の3アカウントで主要な業務フローを確認できます。

■ 申請者アカウント

ログインID: watanabe

パスワード: password123

■ 部門管理者アカウント

ログインID: kobayashi

パスワード: password123

■ 本部管理者アカウント

ログインID:tajiri

パスワード:password123